proyecto „restaurante express"

Parcial Integrador

Materia: Programación I (17047)
Docente: Abel Israel Laime Huanca
Fecha: 13/10/2025

Integrantes:

Rodriguez, Tiago
Martin, Agustina
Herreros, Lucía

índice

Introducción 1

Objetivo 2

Proyecto y contexto 3

Equipo de trabajo 4

Explicación inicial del código 5

- main.py 5
- menu.pu 6
- pedidos.py 7
- reservas.py 9
- platos.json 11

Conclusiones 12

Bibliografía 13

Introducción

En el siguiente proyecto se desarrolla una aplicación de consola que simula el sistema de un restaurante llamado “Restaurante Express”. El programa ofrece un menú con distintas opciones que el usuario puede elegir. Contaremos con algunas funciones claves como por ejemplo:

- La posibilidad de ver carta de platos (con sus respectivos precios y descripción)
- Reservar mesa (nombre, cantidad de personas, fecha y hora)
- Realizar pedidos
- Y poder salir del sistema.

La aplicación busca reproducir de forma básica el funcionamiento administrativo de un restaurante.

Objetivo

Nuestro objetivo principal se centrará en crear una aplicación funcional y organizada ofreciéndole al usuario una experiencia fluida y cómoda para que pueda interactuar sin complicaciones de forma intuitiva y eficiente. Se busca aplicar buenas prácticas mediante la separación lógica del código en funciones y/o clases, garantizando claridad, modularidad y facilidad de mantenimiento. Además, se busca reducir los errores de ingreso de datos mediante validaciones robustas y adecuadas.

Proyecto y contexto

El proyecto propone una aplicación de consola que representa una versión simplificada de un sistema de administración de establecimientos gastronómicos.
El programa se ejecuta directamente desde la terminal y permite al usuario simular procesos clave en el ámbito de un restaurante: consultar menú de platos disponibles con sus precios, realizar reservas y efectuar pedidos.
El proyecto se organizó en una estructura modular que facilita la comprensión, mantenimiento y ampliación del código. Cada archivo cumple una función específica:

- main.py
- menu.py
- pedidos.py
- reservas.py
- platos.json

Enlace al repositorio GitHub

Equipo de trabajo

Para trabajar de forma más organizada y garantizar una gestión efectiva del proyecto, los roles de trabajo se dividieron de la siguiente manera:

- Team Leader: Tiago Rodriguez
    - Coordinación general del proyecto, asegurando el cumplimiento de tareas y objetivos.
- Programador: Agustina Martin
    - Desarrollo de las funcionalidades de la aplicación.
- Tester: Lucía Herreros
    - Verificación del código y corrección de errores e inconsistencias.

Explicación inicial del código

main.py

Es el punto de entrada del programa. Presenta el menú principal por consola, desde donde el usuario puede navegar entre las distintas funcionalidades del sistema (visualizar el menú de platos, realizar pedidos o gestionar reservas).

- Primero se importan los otros módulos para poder llamar a las funciones cuando el usuario elige una opción.
- Se imprimen las opciones y se pide input al usuario para que elija alguna.
- Se utiliza un loop *while*, *try/except* y también condicionales para validar el input del usuario.
- Según la opción que elija se llama a las funciones correspondientes o se sale del sistema.

```
=== RESTAURANTE ===
1. Ver la carta
2. Hacer una reserva
3. Hacer un pedido
4. Ver reservas
5. Salir
Elija una opción: a
⚠️Por favor ingrese un número.


=== RESTAURANTE ===
1. Ver la carta
2. Hacer una reserva
3. Hacer un pedido
4. Ver reservas
5. Salir
Elija una opción: 6
Opción inválida. Ingrese un número entre 1 y 5


=== RESTAURANTE ===
1. Ver la carta
2. Hacer una reserva
3. Hacer un pedido
4. Ver reservas
5. Salir
Elija una opción: 5
¡Hasta pronto!
```

menu.py

Contiene funciones relacionadas con la gestión y visualización de la carta del restaurante. Permite cargar los datos de los platos desde el archivo platos.json y mostrarlos en formato tabulado para una mejor experiencia del usuario.

- Se intenta abrir el archivo platos.json en modo lectura con un bloque *with* para gestionar el cierre del archivo automáticamente. El contenido del archivo se carga en la variable platos.
- Para definir el formato visual de la tabla, se establecen tamaños fijos para las columnas y los encabezados que se mostrarán. Esto permite mantener una estructura alineada sin importar la longitud de los textos.
- Primero se muestra el título del menú, seguido de los encabezados con líneas divisorias. Se utilizan f-strings con alineación centrada para que el texto quede centrados dentro de cada columna.
- Se utiliza una estructura *for* para recorrer la lista de platos y mostrar cada uno en su propia fila. Cada fila se imprime siguiendo el mismo formato previo.
- Para cerrar la tabla se imprime otra línea divisoria.
- Los bloques *try/except* se utilizan para controlar posibles errores:
    - *FileNotFound* si el archivo platos.json no existe
    - *json.JSONDecodeError* si el contenido del archivo no tiene un formato JSON válido.
- Se incluye un bloque finally para garantizar que, si algo falló, el archivo se cierre.

```
--- MENÚ DE PLATOS ---
|--------|-----------------------------|-----------------------------|
|        |            Plato            |            Precio           |
|--------|-----------------------------|-----------------------------|
|   1    |  Milanesa con papas fritas  |            $13500           |
|   2    |       Pizza muzzarella      |            $12800           |
|   3    |        Ensalada Cesar       |            $13000           |
|   4    |           Ravioles          |            $14000           |
|--------|-----------------------------|-----------------------------|
```

pedidos.py

La función hacer_pedido() simula el proceso de realización de un pedido. Incluye la lógica necesaria para seleccionar platos, agregarlos al pedido y calcular el total de la compra.

- Se imprime un título de presentación.
- Se intenta abrir el archivo platos.json en modo lectura con un bloque *with* para gestionar el cierre del archivo automáticamente. El contenido del archivo se carga en la variable platos.
- Se utiliza una estructura *for* para mostrar el menú.
- Se crean dos variables: una para almacenar los platos elegidos en una lista y otra como acumulador para ir sumando los precios de los platos seleccionados.
- Se le solicita al usuario su nombre para identificar el pedido.
- Se utiliza un bucle *while* para que el usuario pueda ingresar varios platos utilizando un ID (o para finalizar el pedido y salir con la palabra “fin”).
- Se valida el input utilizando condicionales para asegurarse de que el usuario ingrese solo números enteros que sean válidos.
- Se recorre la lista de platos para buscar el plato cuyo ID coincida con la opción ingresada por el usuario. Si no se encuentra ningún plato con el ID ingresado, se informa al usuario que ingresó una opción inválida. De lo contrario, se agrega el plato a la lista del pedido y se suma su valor al total de la compra.
- Finalmente se imprime el resumen del pedido, con los nombres de los platos, sus precios y su total. Si no se agregaron platos al pedido, se imprime un mensaje que lo indica.
- Los bloques *try/except* se utilizan para controlar posibles errores:
    - *FileNotFound* si el archivo platos.json no existe
    - *json.JSONDecodeError* si el contenido del archivo no tiene un formato JSON válido.

```
--- HACER UN PEDIDO ---
1. Milanesa con papas fritas - $13500
2. Pizza muzzarella - $12800
3. Ensalada Cesar - $13000
4. Ravioles - $14000

Ingrese su nombre: Ana

Ingrese el ID del plato (o 'fin' para terminar): 1
✅ Milanesa con papas fritas agregado al pedido.

Ingrese el ID del plato (o 'fin' para terminar): 2
✅ Pizza muzzarella agregado al pedido.

Ingrese el ID del plato (o 'fin' para terminar): a
❌ Por favor ingrese un número válido.

Ingrese el ID del plato (o 'fin' para terminar): 6
❌ ID inválido, intente nuevamente.

Ingrese el ID del plato (o 'fin' para terminar): fin

--- RESUMEN DEL PEDIDO ---
Pedido para Ana
- Milanesa con papas fritas ($13500)
- Pizza muzzarella ($12800)
TOTAL: $26300
```

reservas.py

Aquí se administra la funcionalidad de reservas de mesas. La función hacer_reserva() permite al usuario ingresar datos como nombre, fecha, hora y cantidad de personas, simulando el proceso de reserva dentro del sistema del restaurante.

- Se define una lista global para guardar las reservas realizadas durante la ejecución del programa.
- Se imprime un título de presentación.
- Se le informa al usuario de los horarios del restaurante para que no ingresen horarios fuera de los estipulados.
- Se solicita al usuario los datos básicos de la reserva, utilizando validaciones (*try/except* y condicionales) para asegurar que los datos ingresados sean válidos:
    - Nombre del cliente.
    - Cantidad de personas.
    - Fecha de la reserva (formato día/mes).
    - Hora de la reserva (formato hora:minuto).
- Los datos se agrupan en un diccionario, con claves que identifican cada campo, y este se agrega a la lista de reservas.
- Se imprime un mensaje confirmando que la reserva fue registrada correctamente.

La función mostrar_reservas() permite consultar las reservas registradas:

- Si la lista de reservas está vacía, se informa que aún no hay ninguna reserva y la función termina.
- Se recorre la lista con *enumerate* para asignar un número consecutivo a cada reserva. En cada línea se muestra:
    - El número de reserva.
    - El nombre del cliente.
    - La cantidad de personas.
    - La fecha y hora de la reserva.

```
--- HACER UNA RESERVA ---
 ❗ Por favor, tenga en cuenta que los horarios del restaurante son de 19:00 hs
 a 23:59 hs  ❗

Ingrese su nombre: Juan
Cantidad de personas: 4
Fecha (DD/MM): 13/10
Hora (HH:MM): 18:30
❌ Ingrese un horario válido.
Hora (HH:MM): 21:60
❌ Ingrese un horario válido.
Hora (HH:MM): 21:30

✅ Reserva registrada para Juan el 13/10 a las 21:30hs.
```

platos.json

Este archivo almacena la información de los platos del menú (ID, nombre y precio), lo que permite separar la lógica del programa de los datos y facilitar su modificación sin alterar el código fuente. Al ser un archivo persistente se encuentra en una carpeta aparte llamada “datos”.

Conclusiones

El desarrollo de este proyecto permitió aplicar los conocimientos adquiridos sobre programación estructurada y modular en Python, integrando conceptos como el uso de funciones, manejo de archivos, validación de datos, estructuras de control, arreglos multidimensionales, etc. A través de su diseño, se logró simular de manera funcional el sistema básico de gestión de un restaurante.
El trabajo en equipo facilitó la división de tareas y el cumplimiento de los objetivos, evidenciando la importancia de la colaboración y la organización en proyectos de este tipo.
En definitiva, el proyecto representa de manera clara el uso práctico de la programación para resolver necesidades y problemas reales de forma eficiente.

Bibliografía

Bibliografía de la materia
Material de clase
Documentación de Python